

Open Data Chunker

Pipeline ETL ad Alte Prestazioni

Documentazione Tecnica

Versione 1.0

26 maggio 2026

Indice

1	Introduzione	3
1.1	Contesto	3
1.2	Obiettivi del Progetto	3
1.3	Target Audience	4
2	Architettura	5
2.1	Panoramica	5
2.2	Layer di Input	5
2.3	Layer di Processing	5
2.4	Layer di Storage	6
2.5	Layer di Query	6
2.6	Flusso dei Dati	6
3	Implementazione	7
3.1	Stack Tecnologico	7
3.2	Sanitizzazione XML	7
3.3	Parsing Streaming	8
3.4	Parallelismo Multi-Processo	8
3.5	Batching e Scrittura	9
4	Modello Dati	10
4.1	Struttura Gerarchica XML	10
4.2	Normalizzazione in Tre Tabelle	10
4.2.1	Tabella AIUTI	10
4.2.2	Tabella COMPONENTI	10
4.2.3	Tabella STRUMENTI	10
4.3	Diagramma ER	11
4.4	Partizionamento Hive-Style	11
5	Performance	13
5.1	Metriche di Riferimento	13
5.2	Scalabilità Orizzontale	13
5.3	Compressione Parquet	13
5.4	Ottimizzazioni Chiave	14
5.4.1	Streaming Parser	14
5.4.2	Batch Writing	14
5.4.3	Aggressive Cleanup	14

6	Containerizzazione Docker	15
6.1	Vantaggi della Containerizzazione	15
6.2	Dockerfile	15
6.2.1	Ottimizzazioni Docker	16
6.3	Docker Compose	16
6.3.1	Volume Mounting	16
6.4	Workflow Tipico	16
6.5	CI/CD Integration	17
7	CLI e Usabilità	18
7.1	Framework Click	18
7.2	Comandi Disponibili	18
7.2.1	parse	18
7.2.2	query	18
7.2.3	export	19
7.2.4	export-aggregated	19
7.3	Progress Bar	20
7.4	Error Handling	20
8	Inference Service e Classificazione	21
8.1	Architettura del Microservizio	21
8.2	Metodologia di Estrazione e Classificazione	21
9	Conclusioni	23
9.1	Riepilogo Punti di Forza	23
9.2	Possibili Evoluzioni	23
9.3	Considerazioni Finali	24

Capitolo 1

Introduzione

1.1 Contesto

Il **Registro Nazionale degli Aiuti** (RNA) è la banca dati nazionale istituita presso il Ministero dello Sviluppo Economico che raccoglie le informazioni relative agli aiuti di Stato e agli aiuti *de minimis* concessi in Italia. L’RNA pubblica periodicamente dataset in formato **Open Data XML**, consentendo l’accesso pubblico a informazioni dettagliate su milioni di contributi erogati.

Questi dataset presentano caratteristiche che li rendono complessi da gestire:

- **Volumi elevati:** centinaia di file XML, ciascuno contenente decine di migliaia di record
- **Struttura gerarchica:** dati annidati su più livelli (Aiuti → Componenti → Strumenti)
- **Qualità variabile:** presenza di caratteri XML non validi e encoding inconsistente
- **Assenza di formati analitici:** i dati sono distribuiti solo in XML, non ottimale per query analitiche

1.2 Obiettivi del Progetto

Open Data Chunker è una pipeline ETL (*Extract, Transform, Load*) progettata per rispondere a queste sfide. Gli obiettivi principali sono:

1. **Parsing robusto:** capacità di processare file XML corrotti o malformati senza interruzioni
2. **Performance scalabili:** elaborazione parallela multi-processo per sfruttare CPU multi-core
3. **Output analitico:** conversione in formato **Apache Parquet** con partizionamento temporale
4. **Query interattive:** integrazione con motori SQL (**DuckDB**) per interrogazioni ad-hoc
5. **Riproducibilità:** containerizzazione Docker per ambienti consistenti

Punti di Forza

- Pipeline end-to-end: dal raw XML al dataset queryable
- Fault-tolerant: gestione automatica degli errori
- Compressione 10x rispetto all'XML originale
- Deploy immediato via Docker

1.3 Target Audience

Questa documentazione è rivolta a **Software Engineer** e **Data Engineer** interessati a:

- Comprendere l'architettura di una pipeline ETL Python moderna
- Valutare pattern di parallelismo e gestione memoria
- Replicare o estendere la soluzione per casi d'uso simili

Capitolo 2

Architettura

2.1 Panoramica

L'architettura di Open Data Chunker segue un pattern **layered** con separazione netta delle responsabilità. Il sistema è composto da quattro layer principali, illustrati nel diagramma seguente.

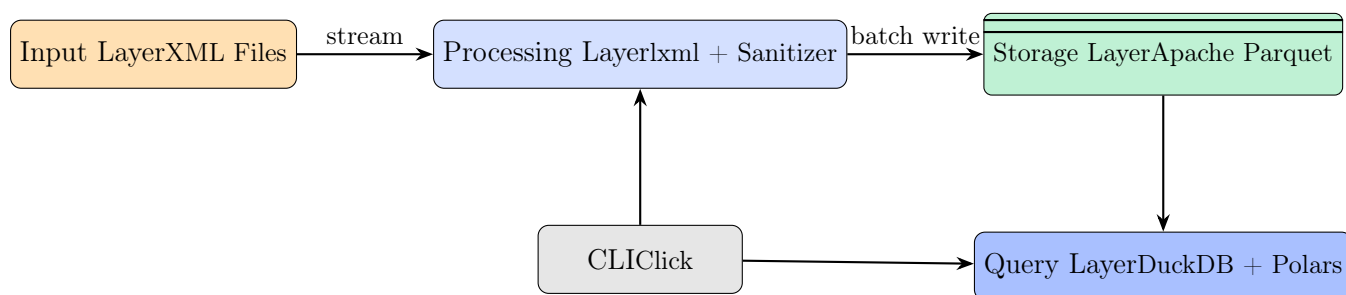


Figura 2.1: Architettura a layer di Open Data Chunker

2.2 Layer di Input

Il layer di input gestisce i file XML grezzi provenienti dall'RNA Open Data. Caratteristiche:

- Supporto per file singoli o directory ricorsive
- Discovery automatico tramite glob pattern `*.xml`
- Nessun caricamento in memoria dell'intero file (streaming)

2.3 Layer di Processing

Il core della pipeline, implementato in `src/parser.py`. Composto da:

Componente	Responsabilità
CleanFileInputStream	Wrapper che filtra caratteri XML non validi in tempo reale
iterparse()	Parser SAX-like per elaborazione streaming
ProcessPoolExecutor	Orchestratore del parallelismo multi-processo
flush_batches()	Writer batch su Parquet con partizionamento

Tabella 2.1: Componenti del Processing Layer

2.4 Layer di Storage

Output in formato **Apache Parquet** con le seguenti caratteristiche:

- **Columnar storage:** compressione ottimale e predicate pushdown
- **Hive partitioning:** directory ANNO=YYYY per partition pruning
- **Schema tipizzato:** definizioni PyArrow in `src/models.py`

Vantaggio: Partition Pruning

Le query filtrate per anno leggono solo le partizioni rilevanti, riducendo I/O fino al 90% su dataset multi-anno.

2.5 Layer di Query

Dual-engine per flessibilità:

DuckDB Motore SQL analitico in-process. Ideale per aggregazioni e join complessi via CLI.

Polars DataFrame library lazy-first. Utilizzato per export streaming memory-efficient.

2.6 Flusso dei Dati

1. L'utente invoca `parse -input data/` via CLI
2. Il CLI discovery tutti i file `*.xml` nella directory
3. I file vengono distribuiti tra N worker processes
4. Ogni worker:
 - (a) Apre il file tramite `CleanFileInputStream`
 - (b) Itera gli elementi `<AIUTO>` con `iterparse()`
 - (c) Estrae i campi in batch da 10.000 record
 - (d) Scrive il batch su Parquet partizionato
5. Al termine, statistiche aggregate vengono loggite

Capitolo 3

Implementazione

3.1 Stack Tecnologico

La soluzione utilizza un moderno stack Python per data engineering:

Libreria	Versione	Utilizzo
lxml	5.x	Parsing XML streaming
polars	1.x	DataFrame processing
pyarrow	18.x	Schema definition e Parquet I/O
duckdb	1.x	Query engine SQL
click	8.x	CLI framework
tqdm	4.x	Progress bar

Tabella 3.1: Dipendenze principali

3.2 Sanitizzazione XML

Un problema critico dei dataset RNA è la presenza di **caratteri XML non validi**. Questi includono:

- Caratteri di controllo ASCII: 0x00-0x08, 0x0B-0x0C, 0x0E-0x1F
- Entità numeriche malformate: � -

La classe `CleanFileInputStream` risolve questo problema tramite un wrapper file-like che filtra lo stream in tempo reale:

```
1 class CleanFileInputStream:
2     def __init__(self, filename):
3         self.f = open(filename, 'rb')
4         self.invalid_xml_chars = re.compile(
5             b'[\x00-\x08\x0b\x0c\x0e-\x1f]|'
6             b'&#(?:[0-8]|1[1-2]|1[4-9]|2[0-9]|3[0-1]);'
7         )
8
9     def read(self, size=-1):
```

```

10     data = self.f.read(size)
11     return self.invalid_xml_chars.sub(b'', data)

```

Listing 3.1: Sanitizzazione streaming

Design Choice

Il pattern wrapper consente di integrare la sanitizzazione senza modificare il codice di parsing. Il parser lxml riceve uno stream già pulito.

3.3 Parsing Streaming

Per gestire file multi-gigabyte senza esaurire la memoria, utilizziamo `lxml.etree.iterparse()` con cleanup aggressivo:

```

1 context = etree.iterparse(
2     clean_stream,
3     events=("end",),
4     tag=f"{NS}AIUTO"
5 )
6
7 for event, elem in context:
8     # Estrazione dati...
9
10    # Release memory
11    elem.clear()
12    while elem.getprevious() is not None:
13        del elem.getparent()[0]

```

Listing 3.2: Iterparse con memory management

Il pattern `elem.clear()` + deletion dei siblings previene memory leak tipici di `iterparse`.

3.4 Parallelismo Multi-Processo

L'elaborazione parallela sfrutta `ProcessPoolExecutor` per bypassare il GIL:

```

1 with ProcessPoolExecutor(max_workers=workers) as executor:
2     futures = {
3         executor.submit(process_file, f, output): f
4         for f in files
5     }
6
7 for future in as_completed(futures):
8     stats = future.result()
9     # Aggregate statistics...

```

Listing 3.3: Orchestrazione parallela

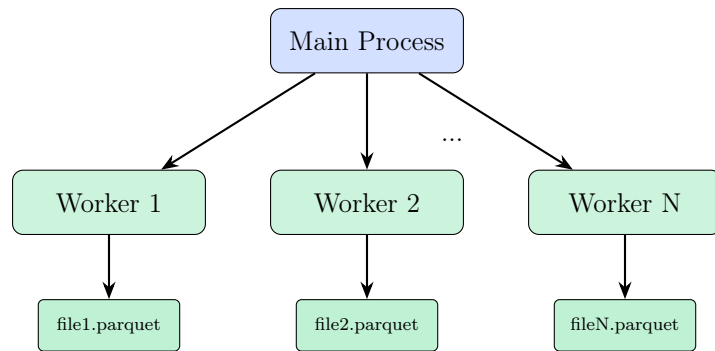


Figura 3.1: Modello fork-join per elaborazione parallela

3.5 Batching e Scrittura

Per ottimizzare le scritture su disco, i record vengono accumulati in batch di 10.000 elementi prima del flush:

```
1 BATCH_SIZE = 10000
2
3 if len(batch_aiuti) >= BATCH_SIZE:
4     pq.write_to_dataset(
5         table,
6         root_path=str(base_path / "aiuti"),
7         partition_cols=['ANNO'],
8         existing_data_behavior='overwrite_or_ignore'
9     )
```

Listing 3.4: Flush batch su Parquet

Il parametro `partition_cols=['ANNO']` abilita il **Hive-style partitioning**, creando automaticamente directory `ANNO=2022/`, `ANNO=2023/`, etc.

Capitolo 4

Modello Dati

4.1 Struttura Gerarchica XML

I file XML dell'RNA presentano una struttura gerarchica a tre livelli:

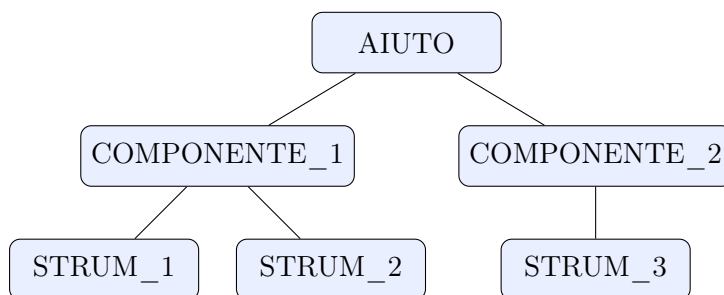


Figura 4.1: Struttura gerarchica dei dati RNA

4.2 Normalizzazione in Tre Tabelle

Il parser trasforma la struttura gerarchica in tre tabelle relazionali normalizzate:

4.2.1 Tabella AIUTI

Entità principale contenente le informazioni sul contributo:

4.2.2 Tabella COMPONENTI

Dettaglio delle componenti di aiuto, in relazione 1:N con AIUTI:

4.2.3 Tabella STRUMENTI

Strumenti finanziari applicati, in relazione 1:N con COMPONENTI:

Campo	Tipo	Descrizione
CAR	string	Codice Aiuto Regionale (PK logica)
COR	string	Codice Operazione Registro
TITOLO_MISURA	string	Denominazione del bando
DATA_CONCESSIONE	string	Data di erogazione
DENOMINAZIONE_BENEFICIARIO	string	Ragione sociale beneficiario
CODICE_FISCALE_BENEFICIARIO	string	CF/P.IVA
REGIONE_BENEFICIARIO	string	Regione di appartenenza
CUP	string	Codice Unico Progetto
ANNO	int32	Partizione temporale
FILE_SOURCE	string	Tracciabilità file sorgente

Tabella 4.1: Schema tabella AIUTI

Campo	Tipo	Descrizione
ID_COMPONENTE_AIUTO	string	Identificativo componente (PK)
CAR_AIUTO	string	FK verso AIUTI.CAR
COR_AIUTO	string	FK verso AIUTI.COR
COD_REGOLAMENTO	string	Regolamento UE applicato
SETTORE_ATTIVITA	string	Codice ATECO
ANNO	int32	Partizione

Tabella 4.2: Schema tabella COMPONENTI

4.3 Diagramma ER

4.4 Partizionamento Hive-Style

L'output è organizzato con **Hive-style partitioning** sulla colonna ANNO:

```

1 public/parquet/
2   aiuti/
3     ANNO=2022/
4       part-0.parquet
5       part-1.parquet
6     ANNO=2023/
7       part-0.parquet
8   componenti/
9     ANNO=2022/
10    ANNO=2023/
11  strumenti/
12    ANNO=2022/
13    ANNO=2023/

```

Listing 4.1: Struttura directory output

Questo layout consente:

- **Partition pruning:** query filtrate per anno leggono solo le partizioni rilevanti
- **Parallelismo:** file distinti possono essere letti concorrentemente

Campo	Tipo	Descrizione
ID_COMPONENTE_AIUTO	string	FK verso COMPONENTI
COD_STRUMENTO	string	Codice strumento finanziario
IMPORTO_NOMINALE	float64	Valore nominale contributo
ELEMENTO_DI_AIUTO	float64	Equivalente sovvenzione
ANNO	int32	Partizione

Tabella 4.3: Schema tabella STRUMENTI



Figura 4.2: Diagramma ER delle tabelle normalizzate

- **Gestibilità:** facile eliminazione o aggiornamento di singoli anni

Capitolo 5

Performance

5.1 Metriche di Riferimento

Le seguenti metriche sono state misurate su un dataset reale dell'RNA composto da circa 165 file XML:

Metrica	Valore
Velocità di parsing	~50.000 record/sec (8 worker)
Utilizzo memoria per worker	~200 MB
Rapporto compressione	~10x vs XML grezzo

Tabella 5.1: Benchmark su dataset RNA reale

5.2 Scalabilità Orizzontale

La performance scala linearmente con il numero di worker fino al numero di core fisici disponibili:

Figura 5.1: Scalabilità throughput vs numero worker

Nota sulle Performance

Il leggero scostamento dalla linearità ideale è dovuto a:

- Overhead di serializzazione/deserializzazione tra processi
- Contesa I/O su disco per scritture Parquet concorrenti
- Scheduling overhead del sistema operativo

5.3 Compressione Parquet

Il formato Parquet offre vantaggi significativi rispetto all'XML:

Aspetto	XML	Parquet
Storage size	1 GB	~100 MB
Tempo lettura completa	>60s	<5s
Query selettive	Full scan	Partition pruning
Tipizzazione	Assente	Nativa

Tabella 5.2: Confronto XML vs Parquet

5.4 Ottimizzazioni Chiave

5.4.1 Streaming Parser

L'utilizzo di `iterparse()` evita di caricare l'intero DOM in memoria:

- Memory footprint costante indipendentemente dalla dimensione file
- Possibilità di processare file multi-gigabyte su macchine con RAM limitata

5.4.2 Batch Writing

La scrittura in batch da 10.000 record bilancia:

- **Overhead I/O:** meno syscall rispetto a scritture singole
- **Memory usage:** batch sufficientemente piccoli da non saturare RAM

5.4.3 Aggressive Cleanup

Il pattern di cleanup post-parsing previene memory leak:

```

1 elem.clear()
2 while elem.getprevious() is not None:
3     del elem.getparent()[0]
```

Senza questo cleanup, `lxml` mantiene riferimenti all'intero documento ancestry.

Capitolo 6

Containerizzazione Docker

6.1 Vantaggi della Containerizzazione

L'utilizzo di Docker garantisce:

- **Riproducibilità:** ambiente identico su qualsiasi macchina
- **Isolamento:** nessun conflitto con librerie di sistema
- **Portabilità:** deploy immediato su cloud, CI/CD, workstation locali
- **Versionamento:** immagini taggate per rollback facili

Best Practice

La containerizzazione elimina il classico problema “works on my machine”. Ogni esecuzione avviene in un ambiente controllato e ripetibile.

6.2 Dockerfile

Il Dockerfile utilizza un'immagine base `python:3.12-slim` per minimizzare la dimensione:

```
1 FROM python:3.12-slim
2
3 WORKDIR /app
4
5 # Install system dependencies
6 RUN apt-get update && apt-get install -y \
7     build-essential \
8     && rm -rf /var/lib/apt/lists/*
9
10 # Copy requirements and install
11 COPY requirements.txt .
12 RUN pip install --no-cache-dir -r requirements.txt
13
14 # Copy source code
15 COPY . .
16
17 # Set python path
```

```

18 ENV PYTHONPATH=/app
19
20 # Default command
21 CMD ["python", "-m", "src.cli"]

```

Listing 6.1: Dockerfile

6.2.1 Ottimizzazioni Docker

Multi-stage non necessario Le dipendenze Python non richiedono compilazione pesante

Cache layer requirements.txt copiato prima del codice per cache efficiente

Cleanup apt Rimozione cache /var/lib/apt/lists/* per immagine più leggera

—no-cache-dir Evita cache pip inutile nel container

6.3 Docker Compose

Il file docker-compose.yml semplifica l'esecuzione:

```

1 services:
2   etl:
3     build: .
4     volumes:
5       - ./data:/app/data
6       - ./public:/app/public
7     environment:
8       - PYTHONUNBUFFERED=1
9     command: ["tail", "-f", "/dev/null"]

```

Listing 6.2: docker-compose.yml

6.3.1 Volume Mounting

Due volumi persistenti garantiscono che:

Volume	Scopo
./data:/app/data	Input XML accessibili dal container
./public:/app/public	Output Parquet persistito su host

Tabella 6.1: Volumi Docker

6.4 Workflow Tipico

```

1 # Build dell'immagine
2 docker compose build
3
4 # Parsing di tutti i file XML
5 docker compose run --rm etl python -m src.cli parse \
6     --input data/ --workers 8
7
8 # Query interattiva
9 docker compose run --rm etl python -m src.cli query \
10     --table aiuti --limit 10
11
12 # Export CSV
13 docker compose run --rm etl python -m src.cli export \
14     --table aiuti --output public/exports/aiuti.csv

```

Listing 6.3: Comandi Docker tipici

6.5 CI/CD Integration

La natura containerizzata facilita l'integrazione in pipeline CI/CD:

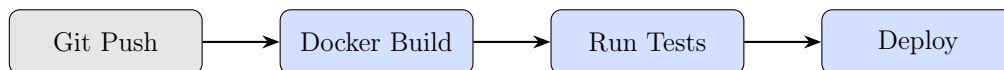


Figura 6.1: Pipeline CI/CD tipica

Capitolo 7

CLI e Usabilità

7.1 Framework Click

L'interfaccia a riga di comando è costruita con **Click**, un framework Python che offre:

- Parsing automatico degli argomenti
- Validazione dei tipi
- Help auto-generato
- Composizione di comandi (gruppi)

7.2 Comandi Disponibili

7.2.1 parse

Elabora file XML e li converte in Parquet:

```
1 docker compose run --rm etl python -m src.cli parse \  
2   --input data/ \  
3   --output public/parquet \  
4   --workers 8
```

Opzione	Default	Descrizione
-i, -input	(required)	File o directory input
-o, -output	public/parquet	Directory output
-w, -workers	4	Numero processi paralleli

Tabella 7.1: Opzioni comando parse

7.2.2 query

Esegue query SQL interattive via DuckDB:

```

1 # Query semplice
2 docker compose run --rm etl python -m src.cli query \
3     --table aiuti --limit 5
4
5 # Query SQL custom
6 docker compose run --rm etl python -m src.cli query \
7     --table strumenti \
8     --query "SELECT ANNO, SUM(IMPORTO_NOMINALE) as tot
9             FROM read_parquet('public/parquet/strumenti/**/*.*.parquet
10             '))
11             GROUP BY ANNO ORDER BY ANNO"

```

Opzione	Default	Descrizione
-t, --table	(required)	Tabella target
-q, --query	—	Query SQL custom
-l, --limit	10	Limite risultati

Tabella 7.2: Opzioni comando query

7.2.3 export

Esporta tabelle in formato CSV o TXT:

```

1 docker compose run --rm etl python -m src.cli export \
2     --table aiuti \
3     --format csv \
4     --output public/exports/aiuti.csv \
5     --delimiter ","

```

7.2.4 export-aggregated

Export specializzato con join e aggregazioni pre-calcolate:

```

1 docker compose run --rm etl python -m src.cli export-aggregated \
2     --output public/exports/aggregated.csv

```

Questo comando:

1. Effettua join tra AIUTI, COMPONENTI e STRUMENTI
2. Aggrega IMPORTO_NOMINALE e ELEMENTO_DI_AIUTO
3. Conta componenti e strumenti per aiuto
4. Produce un file CSV per anno per ottimizzare la memoria

7.3 Progress Bar

Durante l'elaborazione, `tqdm` fornisce feedback visivo in tempo reale:

```
1 Found 165 XML files to process
2 Starting processing with 8 workers...
3 Processing files: 100%|=====| 165/165 [02:15<00:00]
4 Processing completed in 135.42 seconds
5 Total processed records: {'aiuti': 2847293, 'componenti': ...}
```

Listing 7.1: Output con progress bar

7.4 Error Handling

Il sistema gestisce gli errori in modo graceful:

- **File corrotti:** loggati ma non bloccano l'esecuzione
- **failures.txt:** lista dei file falliti salvata per retry
- **Statistiche accurate:** solo file processati correttamente conteggiati

Capitolo 8

Inference Service e Classificazione

8.1 Architettura del Microservizio

Per arricchire il dataset con informazioni relative all'adozione di nuove tecnologie da parte delle imprese, è stato integrato un microservizio di inferenza denominato **Pack-a-Punch**. Questo componente espone funzionalità di *Natural Language Processing* atte a valutare le descrizioni dei progetti per distinguere le iniziative che impiegano, sviluppano o implementano l'Intelligenza Artificiale, da quelle che non la coinvolgono.

L'architettura dell'inference service si distingue per l'elevato livello di ottimizzazione:

- **Modello per Classificazione Binaria:** dbmdz/bert-base-italian-xxl-cased. Sfruttando un modello BERT specificamente pre-addestrato sulla lingua italiana, si ottiene un'elevata precisione nell'interpretazione del contesto semantico nazionale.
- **Modello per Classificazione Multiclasse:** FacebookAI/xlm-roberta-large. Scelto perchè il training set costruito è in lingua inglese; questo modello supporta e gestisce nativamente l'inferenza e l'addestramento multilingua.
- **High-Performance Backend:** Il modello allenato (anche tramite knowledge distillation da grandi LLM) è esportato e servito mediante **PyTorch (compilato con Dynamo)** con supporto all'accelerazione GPU (CUDA). Ciò abbatte i tempi di inferenza a frazioni di millisecondo per iterazione.
- **Esposizione API:** Sviluppato in **FastAPI** esponendo l'endpoint REST POST `/classify`, capace di gestire inferenze in modalità *batch*, riducendo l'overhead di rete tra il layer ETL e il layer di AI.

8.2 Metodologia di Estrazione e Classificazione

A causa dell'ingente volume di record (milioni di aiuti), una classificazione "per riga" avrebbe comportato inefficienze strutturali, specialmente constatando l'elevata ripetitività delle descrizioni. La metodologia implementata utilizza un approccio progressivo a **Multi-livello con Caching**, eseguito in fasi distinte:

1. **Estrazione dei testi univoci:** Sfruttando il motore relazionale DuckDB, vengono interrogate tutte le partizioni Parquet del dataset originale per isolare unicamente le descrizioni distinte nella colonna DESCRIZIONE_PROGETTO.

2. **Fase 1 - Classificazione Binaria:** L'endpoint `/classify` valuta l'elenco testuale deduplicato con il modello in modalità binaria. L'obiettivo è discernere i progetti strettamente legati all'impiego dell'Intelligenza Artificiale da quelli non pertinenti. I risultati, comprensivi di *confidence score*, popolano una memoria temporanea (`classification_cache.parquet`).
3. **Fase 2 - Classificazione Multiclasse:** Intervenendo solo sui testi identificati con il tag "AI" al passaggio precedente, il layer ETL li instrada nuovamente verso il servizio di classificazione per individuarne lo specifico ambito applicativo avvalendosi dell'inferenza multiclasse. L'output genera una seconda cache stratificata Parquet (`multiclass_cache.parquet`).
4. **Aggregazione Finale (Double Left Join):** Infine, le utility contenute nel modulo `exporter.py` ricongiungono i dataset primari, aggregati parallelamente in chunk annuali con `ThreadPoolExecutor`, mediante un doppio Join con le due cache pre-costruite. I record passati al modello binario ma che sono risultati non inerenti all'AI confluiranno nell'esportazione CSV presentando i dati multiclasse settati al valore neutro di gestione (NULL / UNKNOWN).

Questo approccio consolida una metodologia predittiva altamente efficiente: interrogando il modello per la classificazione complessa solamente quando quella binaria accerta il coinvolgimento dell'Intelligenza Artificiale nel progetto, le performance crescono vertiginosamente evitando computazione disperata e irrilevante.

Capitolo 9

Conclusioni

9.1 Riepilogo Punti di Forza

Open Data Chunker si distingue per i seguenti punti di forza:

1. Robustezza

- Sanitizzazione automatica XML malformati
- Fault tolerance con logging granulare
- Recovery automatico da errori di parsing

2. Performance

- Parsing streaming memory-efficient
- Parallelismo multi-processo scalabile
- Output Parquet con compressione 10x

3. Riproducibilità

- Containerizzazione Docker completa
- Ambiente consistente su qualsiasi piattaforma
- Integrazione CI/CD ready

4. Usabilità

- CLI intuitiva con Click
- Query SQL interattive con DuckDB
- Export flessibile in formati standard

9.2 Possibili Evoluzioni

La soluzione può essere estesa in diverse direzioni:

Incremental Processing Supporto per elaborazione differenziale di soli nuovi file, evitando re-processing completo.

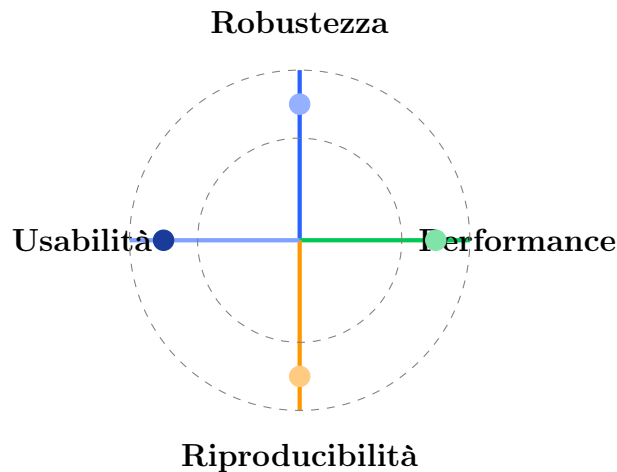


Figura 9.1: I quattro pilastri di Open Data Chunker

Schema Evolution Gestione automatica di cambiamenti nello schema XML sorgente con migrazione trasparente.

Data Quality Integrazione di framework di data quality (es. Great Expectations) per validazione automatica.

Orchestrazione Integrazione con Apache Airflow o Prefect per scheduling e monitoring.

API REST Esposizione dei dati via API per integrazioni applicative.

9.3 Considerazioni Finali

Open Data Chunker dimostra come un'architettura ben progettata possa trasformare dataset pubblici complessi in risorse analitiche accessibili. La combinazione di:

- Modern Python data stack (Polars, DuckDB, PyArrow)
- Pattern di parallelismo robusti
- Containerizzazione per portabilità

...rende la soluzione adatta sia per analisi esplorative occasionali che per pipeline di produzione.

“La qualità dei dati aperti è tanto importante quanto la loro disponibilità.”